



SSD

Solid-State Storage (SSD) Overview

- Replaces traditional Hard Disk Drives (HDDs)
- Built from transistors, no moving parts
- Retains data even without power → persistent storage
- Based on NAND Flash (invented by Fujio Masuoka, 1980s)
- Challenges:
 - Must erase large blocks before writing pages
 - Limited write endurance, cells wear out over time

Storing a Single Bit

- Flash cells store bits using charge levels in a single transistor
- SLC (Single-Level Cell):
 - Stores 1 bit per cell (0 or 1)
 - Fastest, most durable, but expensive
- MLC (Multi-Level Cell):
 - Stores 2 bits per cell (00, 01, 10, 11)
 - Higher density, slower, less durable
- TLC (Triple-Level Cell):
 - Stores 3 bits per cell
 - Lowest cost per bit, but least performance & endurance
- Trade-off: Capacity ↑, Performance & lifespan ↓

Organization

- Flash chips are organized into banks (planes), collections of many cells
- Bank structure:
 - Blocks (erase blocks): 128–256 KB
 - Pages: a few KB (e.g., 4 KB)
 - Each block contains many pages
- Different from disk blocks or memory pages
- Critical property:
 - To write to a page, the entire block must be erased first
 - Affects performance, endurance, and SSD design complexity

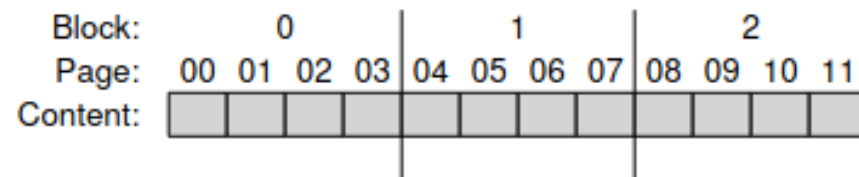


Figure 44.1: A Simple Flash Chip: Pages Within Blocks

Why a Whole Block Must Be Erased Before Writing

- Flash cells store data as charge in floating-gate transistors
- Writing adds charge, but removing charge (erasing) needs high voltage
- Erase operation affects all cells in a block — not individual pages
- Due to shared wiring (word/bit lines), erase can't target one page
- Result:
 - Must erase entire block before rewriting any page
 - Adds complexity, affects performance and endurance

Basic Flash Operations

1. Read (Page)

- Reads a page (2–4 KB) directly
- Fast (tens of μs)
- Random access — uniform speed across device

2. Erase (Block)

- Must erase entire block before writing
- Sets all bits to 1 → destroys old data
- Slow (few ms), enables reprogramming

3. Program (Page)

- Writes new data by changing 1s → 0s
- Slower than read, faster than erase (hundreds μs)
- Page States:
 - Invalid → Erased → Valid
 - Can't reprogram a page without erasing its whole block

		1111	Initial: pages in block are invalid (1)
Erase()	→	EEEE	State of pages in block set to erased (E)
Program(0)	→	VEEE	Program page 0; state set to valid (V)
Program(0)	→	error	Cannot re-program page after programming
Program(1)	→	VVEE	Program page 1
Erase()	→	EEEE	Contents erased; all pages programmable

Example

- Consider a 4-page block and each page is of 8-bit
- To write to page 0, we must erase the entire block, leaving all pages in the erased state.
- We can now program page 0 with new contents (e.g., 00000011), updating the block accordingly.
- The previous contents of pages 1–3 are gone! Before overwriting a page, we must first move any important data elsewhere. Erase behavior strongly affects SSD design.

Page 0	Page 1	Page 2	Page 3
00011000	11001110	00000001	00111111
VALID	VALID	VALID	VALID

Page 0	Page 1	Page 2	Page 3
11111111	11111111	11111111	11111111
ERASED	ERASED	ERASED	ERASED

Page 0	Page 1	Page 2	Page 3
00000011	11111111	11111111	11111111
VALID	ERASED	ERASED	ERASED

Flash Memory Summary

- Reading: Easy and fast; flash can far exceed HDD random read speeds.
- Writing: Tricky; must erase entire block first and move important data.
- Challenges:
 - Program/erase cycles are slow and cause wear-out.
 - Flash performance & reliability are critical in storage design.
- Modern SSDs: Use techniques to deliver high performance and reliability despite these limits.

Device	Read (μ s)	Program (μ s)	Erase (μ s)
SLC	25	200-300	1500-2000
MLC	50	600-900	~3000
TLC	~75	~900-1350	~4500

Figure 44.2: Raw Flash Performance Characteristics

Flash Performance & Reliability

- Read: Fast (tens of microseconds).
- Program (write): Slower, variable; faster with multiple chips in parallel.
- Erase: Expensive (milliseconds).
- Reliability:
 - Main issue: Wear out from repeated program/erase cycles.
 - MLC: ~10,000 P/E cycles
 - SLC: ~100,000 P/E cycles
- Other issue: Disturbance (bit flips in neighboring pages during read/write).
- Design Focus: Mitigating write/erase cost and wear is central to modern SSD design.

Turning Raw Flash into an SSD

- Provide standard block-based interface (512B+ blocks) on top of raw flash.
- **Raw Flash Chips:** Store data but require low-level management (erase/program).
- **SSD Components:**
 - Flash chips → persistent storage
 - Volatile memory (SRAM) → caching & mapping tables
 - Control logic → manages operations

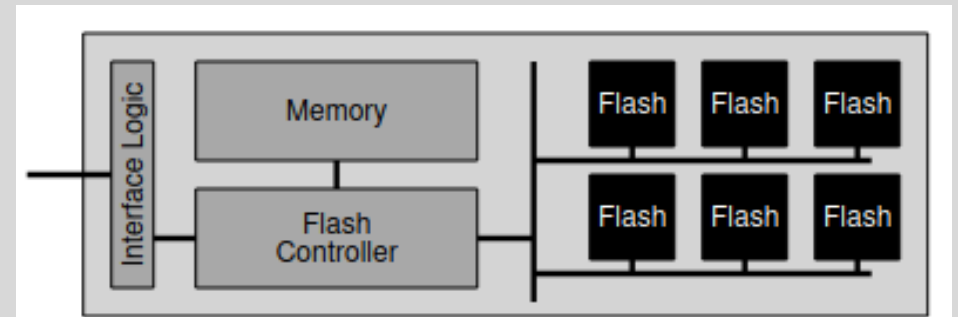


Figure 44.3: A Flash-based SSD: Logical Diagram

FTL – Bridging Client Requests to Flash

- Converts client read/write requests → flash operations (read, erase, program)
- Objectives:
 - High Performance → Use multiple flash chips in parallel
 - High Reliability → Handle wear and program disturbances\
- Performance:
 - Parallelism: Multiple chips → higher throughput
 - Reduce Write Amplification:
$$\text{Write Amplification} = \text{Total flash writes} \div \text{Client writes}$$
- Reliability:
 - Wear Leveling: Spread writes evenly across blocks
 - Sequential Programming: Program pages low → high in a block to minimize disturbances

FTL Organization – Direct-Mapped (Bad Approach)

- Concept:

- Logical page N → Physical page N
- Write to page N requires read entire block → erase → reprogram block

- Performance Issues:

- High write amplification (proportional to block size)
- Slow writes → worse than HDDs in some cases

- Reliability Issues:

- Popular pages cause repeated erases → rapid wear-out
- Client workload directly affects block lifespan

- Direct-mapped FTL is inefficient and unreliable

A Log-Structured FTL

- Log-structured writes: New data is written sequentially to the next available free page in the current block.
- Mapping table: Maintains a mapping of logical block addresses (LBAs) → physical pages.
- Wear leveling: Distributes writes evenly across blocks to extend SSD lifetime.
- Sequential programming: Writes pages from low to high within a block to reduce program disturbance.

Log-Structured FTL Example

- SSD is presented to the client like a regular disk (reads/writes in 512-byte sectors).
- For simplicity let us assume:
 - Client reads/writes 4 KB chunks
 - SSD contains 16 KB blocks, each with 4 pages (4 KB each)
 - Client Write Sequence:
 - Write(100) → contents a1
 - Write(101) → contents a2
 - Write(2000) → contents b1
 - Write(2001) → contents b2

Example

- Client uses **Logical Block Addresses (Eg. 100)** to reference data.
- FTL **translates LBAs → physical pages** on the SSD.
- **Initial state:** All SSD blocks are **INVALID (i)** and must be **erased before writing**.
- Each write involves:
 - Finding a free page
 - Erasing the block (if needed)
 - Programming the page with new data
 - Updating the **mapping table**

[illegible]

Example: Writing Logical Block 100

- Logical Block 100 → Physical Block 0 (Pages 0–3)
- Block 0 is not erased, so the SSD must erase the block first.
- After erase, all pages in Block 0 are ready to be programmed.

Block:	0				1				2			
Page:	00	01	02	03	04	05	06	07	08	09	10	11
Content:												
State:	E	E	E	E	i	i	i	i	i	i	i	i

- FTL will write the data (a1) to the first free page (Page 0).

Block:	0				1				2			
Page:	00	01	02	03	04	05	06	07	08	09	10	11
Content:	a1											
State:	V	E	E	E	i	i	i	i	i	i	i	i

Mapping Logical $\rightarrow$ Physical for Reads

- How to read of **Logical Block 100**?
 - FTL uses the **mapping table** to locate the data.
- Logical Block 100 → Physical Page 0 in Block 0
- **Mapping Table:** Updated when write occurs, stores **LBA → Physical Page**
- **Process:**
 - Client issues read to LBA 100
 - FTL looks up mapping table
 - Reads data from **Physical Page 0**

Table:	100	→	0																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																	
--------	-----	---	---	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

-Cont

- FTL picks the next free page for each write.
- Each write is programmed sequentially within a block.
- Mapping table is updated for each logical block → physical page mapping.
- Reads use the mapping table to locate data.
- Let's now examine the rest of the writes in our example write stream:

101, 2000, and 2001. After writing these blocks, the state of the device is

Table:	100	→	0	101	→	1	2000	→	2	2001	→	3	Memory
Block:	0				1				2				Flash Chip
Page:	00	01	02	03	04	05	06	07	08	09	10	11	
Content:	a1	a2	b1	b2									
State:	V	V	V	V	i	i	i	i	i	i	i	i	

Pros of Log-Structured FTL

- **Improved Performance:**

- Erases required infrequently
- Avoids costly read-modify-write of direct-mapped FTL

- **Enhanced Reliability:**

- Writes are spread across all pages → wear leveling
- Increases device lifetime

- Log-structured approach balances **speed** and **durability** in SSDs

Cons of Log Structured FTL

- **Garbage Accumulation:**

- Overwrites leave **old versions** of data → consume space
- Requires **Garbage Collection (GC)**
- Excessive GC → **higher write amplification**, lower performance

- **Memory Overhead:**

- Large devices need **big in-memory mapping tables**
- Memory requirement grows with device size

- Requires **managing garbage and memory efficiently**

Garbage Collection

- **Overwrites create garbage:** old versions of logical blocks remain on flash
- **Example:**
 - Logical blocks **100 & 101** rewritten as **c1 & c2**
 - Written to **next free pages** (e.g., physical pages 4 & 5)
 - **Mapping table updated** to point to new locations
- **Requirement:**
 - Block 1 must be **erased first** to allow new writes
- **Concept:**
 - Garbage collection (GC) needed to **reclaim space** for future writes

- Physical pages **0 & 1** still contain old versions of blocks 100 & 101
- Pages are **marked VALID**, but data is **obsolete**
- Device must perform **garbage collection (GC)** to reclaim space
- Ensures **free pages** are available for future writes
- Log-structured writing improves performance but **requires managing garbage**

Table:	100	→	4	101	→	5	2000	→	2	2001	→	3	Memory
Block:	0				1				2				
Page:	00	01	02	03	04	05	06	07	08	09	10	11	
Content:	a1	a2	b1	b2	c1	c2							
State:	V	V	V	V	V	V	E	E	i	i	i	i	Flash Chip

Garbage Collection

- The process of finding garbage blocks (also called dead blocks) and reclaiming them for future use is called garbage collection
- Process
 - find a block that contains one or more garbage pages,
 - read in the live (non-garbage) pages from that block,
 - write out those live pages to the log,
 - reclaim the entire block for use in writing.

Example

- Read live data (pages 2 and 3) from block 0
- Write live data to end of the log
- Erase block 0 (freeing it for later usage)
- Garbage collection can be expensive, as it may require reading and rewriting live data.
- Ideal block for GC: a block with only dead pages can be erased and reused immediately.

Table:	100 → 4	101 → 5	2000 → 2	2001 → 3	Memory									
Block:	0				1	2								
Page:	00	01	02	03	04	05	06	07	08	09	10	11		
Content:	a1	a2	b1	b2	c1	c2								
State:	V	V	V	V	V	V	E	E	i	i	i	i		
														Flash Chip

Table:	100	→	4	101	→	5	2000	→	6	2001	→	7	Memory
Block:	0				1				2				Flash Chip
Page:	00	01	02	03	04	05	06	07	08	09	10	11	
Content:					c1	c2	b1	b2					
State:	E	E	E	E	V	V	V	V	i	i	i	i	

Overprovisioning: Add extra flash capacity to **delay cleaning** and run it in the background.

- **Benefits:**

- Minimizes impact on **client-perceived performance**
- Increases **internal bandwidth** for cleaning operations

- **Outcome:** Helps SSDs achieve **excellent overall performance**

Mapping Table Size

- **Large mapping tables are required:** Log-structured, page-level FTL (Flash Translation Layer) needs one mapping entry for each 4-KB page of the device.
- **High memory overhead:** For a large SSD (e.g., 1 TB), storing a 4-byte entry per 4-KB page can require around 1 GB of memory just for the mapping, making this scheme impractical.
- **Three Techniques**
 - Block-level mapping
 - Hybrid mapping
 - Page mapping with caching

Block-Based Mapping – Overview

- Reduce memory needed for FTL mapping
 - Instead of mapping **each page**, map **one pointer per block**
 - Memory reduction factor: $\text{Size_block} / \text{Size_page}$
 - Analogy: Like using **larger page sizes** in virtual memory
- **Pros:**
 - Greatly reduces mapping table size
- **Cons:**
 - Small writes (< block size) require reading and rewriting entire block
 - Leads to **high write amplification** → poor performance

Example

- Assume logical blocks 2000, 2001, 2002, and 2003 (with contents, a, b, c, d), located at physical block 1 at physical pages 4, 5, 6, and 7.
 - With per-page mappings, the translation table would have to record four mappings for these logical blocks:
 - 2000→4, 2001→5, 2002→6, 2003→7.
- FTL stores **one mapping per block** instead of per page
- Logical address divided into:
 - **Chunk number** → identifies the block
 - **Offset** → identifies page within the block
- Example: 4 logical blocks per physical block → **2-bit offset**, Reduces mapping table size significantly
- Logical blocks 2000–2003 share the **same chunk number (500)** but have **different offsets (0–3)**.
- Block-level mapping stores a **single entry**: Chunk 500 → Physical Block 1 (starting at page 4).

Table:	500 → 4												Memory
Block:	0				1				2				Flash Chip
Page:	00	01	02	03	04	05	06	07	08	09	10	11	
Content:					a	b	c	d					
State:	i	i	i	i	V	V	V	V	i	i	i	i	

Block-Based FTL – Read Operation

- **Extract chunk number** from the logical address (topmost bits).
- **Look up mapping table** → get physical page of the block.
- **Add offset** from logical address → get exact physical page.
- **Example:** Logical address: 2002
 - Chunk number: 500 → maps to physical page 4
 - Offset: 2 → physical page = 4 + 2 = 6
 - Data at **physical page 6** is returned.

Table:	500 → 4												Memory
Block:	0				1				2				Flash Chip
Page:	00	01	02	03	04	05	06	07	08	09	10	11	
Content:					a	b	c	d					
State:	i	i	i	i	V	V	V	V	i	i	i	i	

Block-Based FTL – Write Operation

- Writing a single logical block (e.g., 2002 -> c') requires:
 - Reading all other blocks in the physical block (2000, 2001, 2003)
 - Writing all blocks to a **new physical location**
 - Updating the mapping table
 - Erasing the old block
- Block-level mapping **reduces memory overhead** but **causes performance issues** for small writes.
- Real physical blocks can be **256 KB or larger**, so small writes are common.
- **Implication: A better mapping solution** is needed to improve performance.

Table: 500 → 8									Memory			
Block:	0				1				2			
Page:	00	01	02	03	04	05	06	07	08	09	10	11
Content:									a	b	c'	d
State:	i	i	i	i	E	E	E	E	V	V	V	V

Flash Chip

Hybrid Mapping

- Combines block-level and page-level mapping to balance memory use and performance.
- Log blocks: few erased blocks reserved for flexible writing.
- Mapping tables:
 - Log table: per-page mapping for log blocks
 - Data table: per-block mapping for regular blocks
- Lookup process:
 - Check log table first
 - If not found, check data table
- Keep log blocks small to reduce mapping overhead.
- Periodically merge log blocks into regular block mapping.
- Merge techniques: Switch Merge, Partial Merge, Full Merge

Log Table:	1000→0	1001→1	1002→2	1003→3	
Data Table:	250 →8				Memory

Block:	0				1				2				
Page:	00	01	02	03	04	05	06	07	08	09	10	11	Flash
Content:	a'	b'	c'	d'					a	b	c	d	Chip
State:	V	V	V	V	i	i	i	i	V	V	V	V	

Example: Switch Merge (Best Case)

- Initial state: logical pages 1000-1003 in physical block 2 (pages 8-11, data a-d).
- Client overwrites all pages with a', b', c', d' in log block 0 (pages 0-3).
- **Switch Merge:**
 - Log block 0 becomes the new data block.
 - Old block 2 erased and reused as a log block.
 - **Per-page pointers replaced by a single block pointer.**

Log Table:
Data Table: 250 → 8

Block:	0				1				2			
Page:	00	01	02	03	04	05	06	07	08	09	10	11
Content:									a	b	c	d
State:	i	i	i	i	i	i	i	i	V	V	V	V

Flash Chip

Log Table:	1000 → 0	1001 → 1	1002 → 2	1003 → 3	
Data Table:	250 → 8				Memory

Block:	0				1				2				
Page:	00	01	02	03	04	05	06	07	08	09	10	11	
Content:	a'	b'	c'	d'					a	b	c	d	Flash Chip
State:	V	V	V	V	i	i	i	i	V	V	V	V	

Log Table:
Data Table: 250 → 0

Block:	0				1				2			
Page:	00	01	02	03	04	05	06	07	08	09	10	11
Content:	a'	b'	c'	d'								
State:	V	V	V	V	i	i	i	i	i	i	i	i

Flash Chip

Partial Merge

- Only some pages are overwritten (e.g., 1000 and 1001).
- FTL must:
 - Read remaining pages (1002 and 1003) from old block
 - Append them to log block
 - Result: same final state as switch merge, but **extra I/O** increases write amplification.

Log Table: 1000 $\rightarrow$ 0 1001 $\rightarrow$ 1
 Data Table: 250 $\rightarrow$ 8

Block:	0				1				2				
Page:	00	01	02	03	04	05	06	07	08	09	10	11	
Content:	a'	b'							a	b	c	d	
State:	V	V	i	i	i	i	i	i	V	V	V	V	

Flash Chip

Full Merge (Worst Case)

- Log block contains non-contiguous pages
 - **Example: Log Block A** contains pages for logical blocks: 0, 4, 8, 12).
- FTL must:
 - Collect missing pages from multiple blocks
 - Reconstruct full data blocks
- To make it a regular block, FTL must:
 - Collect missing pages for 0–3 from other blocks → write them together.
 - Collect missing pages for 4–7 from other blocks → write them together.
 - Repeat for 8–11, 12–15... until log block A can be erased and reused.
- High **write amplification** → performance penalty
- Frequent full merges should be avoided.

Page Mapping + Caching

- Stores only active logical-to-physical page mappings in memory.
- Reduces memory overhead compared to full page-mapped FTL.
- Frequently accessed pages stay in cache → fast reads/writes.
- If a mapping is not in cache, it is read from flash first → extra read.
- Eviction: Old cached mappings may be removed; if dirty, extra write needed.
- Works best when workload has locality (accesses a small set of pages repeatedly).
- Poor performance if workload is large/random → frequent cache misses.

Wear Leveling in SSDs

- Flash blocks **wear out** after many erase/program cycles.
- FTL spreads writes across all blocks to **avoid “hot spots.”**
- Prevents a few blocks from **failing early** while others remain unused.
- Log-structured writing and garbage collection help **distribute writes initially.**
- Long-lived data blocks may **avoid overwrites**, leading to uneven wear.
- FTL periodically **moves data** from such blocks to new blocks → frees them.
- **Drawback:** Extra I/O increases **write amplification** and reduces performance.

SSD Performance

- SSDs have **no mechanical parts**, behave like **random-access memory (RAM)**.
- **Random I/O**: SSDs vastly outperform HDDs (tens–hundreds MB/s vs ~ 2 MB/s).
- **Sequential I/O**: SSDs faster, but HDDs still perform reasonably well.
- **Random writes**: SSDs perform better than expected due to **log-structured FTL**, which converts random writes into sequential writes.
- **Random reads**: Slightly slower than random writes, but still much faster than HDDs.
- **File system design**: Sequential vs random differences matter; traditional HDD optimizations still relevant, though less critical.

Summary

- **Operating System Drives:** Installing the OS on an SSD can significantly reduce boot times and improve system responsiveness.
- **Gaming:** Faster load times and smoother gameplay experiences.
- **Professional Applications:** Enhanced performance for video editing, 3D rendering, and other resource-intensive tasks.
- **Limitations**
 - **Cost:** Per GB, SSDs are more expensive than HDDs, though prices have been decreasing over time.
 - **Write Endurance:** NAND flash cells have a limited number of write cycles, but modern wear-leveling algorithms have mitigated this issue. [IBM](#)



Thank You